only about twice as big as the second-largest probability. When the temperature approaches infinity, the probability distribution will be uniform (i.e., all probabilities are equal). When the temperature approaches 0, the probability distribution will become a *degenerate distribution* where all the probability mass is at a single point. By using this as a policy function, when $\tau \to \infty$, the actions will be selected completely randomly, and when $\tau \to 0$, the policy becomes the `argmax` function (which we used in the previous section with the 1D Ising model).

The reason this parameter is called "temperature" is because the softmax function is also used in physics to model physical systems like the spins of a system of electrons, where the temperature changes the behavior of the system. There's a lot of cross-pollination between physics and machine learning. In physics it's called the *Boltzmann distribution*, where it "gives the probability that a system will be in a certain state as a function of that state's energy and the temperature of the system" (Wikipedia). In some reinforcement learning academic papers you might see the softmax policy referred to as the Boltzmann policy, but now you know it's the same thing.

We are using a reinforcement learning algorithm to solve a physics problem, so the temperature parameter of the softmax function actually corresponds to the temperature of the electron system we are modeling. If we set the temperature of the system to be very high, the electrons will spin randomly and their tendency to align to neighbors will be overcome by the high temperature. If we set the temperature too low, the electrons will be stuck and won't be able to change much. In listing 9.10 we introduce a function to find the coordinates of agents and another function to generate the rewards in the new 2D environment.

Listing 9.10. Mean field Q-learning: Coordinate and reward functions

```
def get_coords(grid,j):                                          1
    x = int(np.floor(j / grid.shape[0]))                         2
    y = int(j - x * grid.shape[0])                               3
    return x,y

def get_reward_2d(action,action_mean):                           4
    r = (action*(action_mean-action/2)).sum()/action.sum()       5
    return torch.tanh(5 * r)                                     6
```